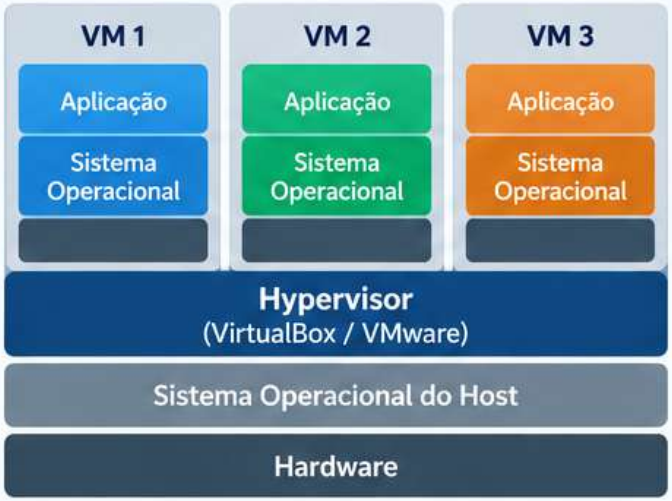
Infocotidiano
CONHECIMENTO QUE TRANSFORMA

Docker x VirtualBox/VMware
Qual a diferença?

Tecnologia na prática

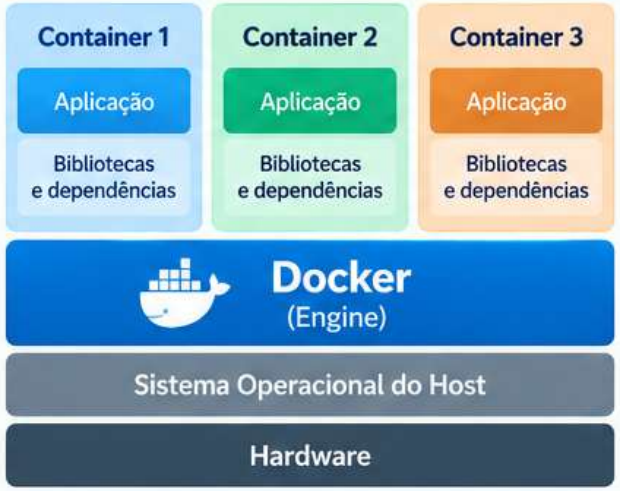
Se você já conhece o VirtualBox ou VMware, entender o Docker fica mais fácil!

VirtualBox / VMware
Máquina Virtual (VM)



- Cada VM tem seu próprio sistema operacional (ex.: Windows, Linux).
- Mais recursos (CPU, RAM, disco).
- Inicialização mais lenta.
- Ideal quando você precisa de um SO completo.

Docker
Containers



- Compartilha o sistema operacional do host.
- Cada container tem apenas a aplicação e suas dependências.
- Utiliza menos recursos.
- Inicialização muito mais rápida.
- Ideal para desenvolvimento, testes e implantar aplicações.

Analogia direta
Para quem já usa VirtualBox/VMware

VirtualBox / VMware

É como ter **vários computadores completos** dentro do seu computador, cada um com seu próprio sistema operacional.

Docker

É como executar **várias aplicações isoladas** no mesmo sistema operacional, sem precisar de um sistema operacional para cada uma.

Em resumo

Característica	VirtualBox/VMware (Máquina Virtual)	Docker (Container)
Sistema Operacional	Cada VM tem o seu	Compartilha o do host
Recursos	Mais recursos	Mais leve
Inicialização	Mais lenta	Mais rápida
Isolamento	SO completo	Aplicação isolada
Uso mais comum	Testar sistemas, ambientes completos	Desenvolvimento, testes e deploy de apps

Objetivo final

Ambas as tecnologias resolvem problemas diferentes.
O Docker não substitui o VirtualBox/VMware, ele complementa!

"Mais conhecimento, mais possibilidades."

Infocotidiano
CONHECIMENTO QUE TRANSFORMA

Infocotidiano
CONHECIMENTO QUE TRANSFORMA

Docker x Container
Qual a diferença?

Tecnologia na prática

De forma simples:

O Docker é a ferramenta (plataforma) e o container é o ambiente isolado que executa sua aplicação.

Docker — A ferramenta que gerencia os containers

Docker

- Plataforma que permite criar, executar e gerenciar containers.
- Faz o download das imagens (por exemplo, do Docker Hub).
- Controla rede, volumes, segurança e outros recursos.
- Funciona no Windows, Linux e macOS.

Pense no Docker como o **"motor"** que faz tudo funcionar, gerenciando os containers.

Container — O ambiente isolado da sua aplicação



- É um ambiente isolado que executa uma aplicação.
- Contém tudo que a aplicação precisa (sistema, bibliotecas, configurações, etc).
- É leve e inicializa rapidamente.
- Pode ser criado, iniciado, parado e removido pelo Docker.

Pense no container como uma **"caixa"** que contém tudo o **que a sua aplicação** precisa para rodar.

Analogia do mundo real:



Docker = Caminhão
O caminhão é a ferramenta que transporta e gerencia as cargas.



Container = Carga
Cada container é uma carga (caixa) com uma aplicação dentro, pronta para ser transportada e usada.

Conhecimento que transforma

Aprenda | Pratique | Evolua | Tecnologia na prática

Infocotidiano
CONHECIMENTO QUE TRANSFORMA

Infocotidiano
compartilhando conhecimento

Da ISO ao Container

Mesma ideia, mundos diferentes.

Se você já usou o DAEMON Tools, entender o Docker fica mais fácil.

1. Arquivo ISO

Imagem do sistema



A ISO contém todo o sistema operacional e seus arquivos.

2. DAEMON Tools

Programa que monta a imagem



DAEMON Tools Lite

Lê o arquivo ISO e cria uma unidade virtual no seu PC.

3. Unidade virtual

A imagem é montada



Unidade D: (ISO)

A ISO é montada como uma unidade de CD/DVD no seu computador.

4. Sistema em execução

Você usa normalmente



O sistema operacional fica disponível para uso, como se fosse um CD/DVD real.

Resultado:

Um sistema operacional funcionando no seu computador sem precisar gravar a ISO.

---

1. Imagem Docker

Modelo da aplicação



postgres:16

A imagem contém o PostgreSQL e tudo que ele precisa para funcionar.

2. Docker (ou Docker Compose)

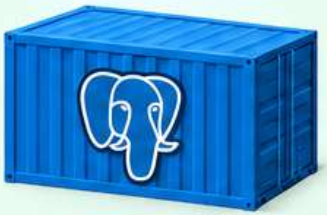
Ferramenta que cria e executa



Lê a imagem e cria um container com as configurações definidas (pode usar Docker Compose).

3. Container

A imagem é executada



postgres-dev

A imagem é executada em um container, que é o ambiente onde o PostgreSQL roda.

4. PostgreSQL em funcionamento

Pronto para usar



O PostgreSQL fica em execução dentro do container, pronto para receber conexões.

Resultado:

O PostgreSQL rodando em um container, isolado, leve e pronto para usar.

Em resumo: Assim como o DAEMON Tools monta uma ISO e cria uma unidade virtual, o Docker usa uma imagem e cria um container onde a aplicação roda.

Infocotidiano
compartilhando conhecimento

Infocotidiano
compartilhando conhecimento

Docker Compose

docker-compose.yml

Define e coordena os containers da aplicação.

O Docker Compose sobe vários containers juntos, configurando rede, portas, volumes, etc.

Container Python (sua aplicação)

Imagem construída com Dockerfile

Rede do Docker

(os containers se comunicam pelo nome do serviço)

Container PostgreSQL (banco de dados)

Imagem pronta do Docker Hub

Dockerfile

```
1  FROM python:3.12
2  WORKDIR /app
3  COPY requirements.txt .
4  RUN pip install -r requirements.txt
5  COPY . .
6  CMD ["python", "app.py"]
```

O Dockerfile define como construir a imagem da sua aplicação Python.

docker-compose.yml

```
1  services:
2    app:
3      build: .          # usa o Dockerfile da pasta
4      ports:
5        - "8000:8000"
6      depends_on:
7        - postgres
8    postgres:
9      image: postgres:16   # imagem pronta do Docker Hub
10     environment:
11       POSTGRES_DB: minha_base
12       POSTGRES_USER: postgres
13       POSTGRES_PASSWORD: senha
```

O Compose usa a imagem que você criou para o Python e uma imagem pronta do PostgreSQL, e configura tudo para funcionar em conjunto.

Dockerfile constrói a imagem. Docker Compose orquestra os containers.
Serviços separados, mais simples, organizados e fáceis de manter.

Infocotidiano
CONHECIMENTO QUE TRANSFORMA

Docker Compose

Principais comandos e o que eles fazem

Tecnologia na prática

Entenda a diferença entre os comandos e como eles afetam seus containers e dados.

Comando	O que faz	Container	Volume/Dados
`docker compose start`	Liga containers que já existem.	Mantém	Mantém
`docker compose down`	Para e **remove** os containers.	Remove	Mantém, normalmente
`docker compose up -d`	Cria (se necessário) e inicia.	Cria/inicia	Mantém

Boas práticas

- Utilize volumes para persistir seus dados.
- Use `docker compose down` apenas quando realmente precisar remover os containers.
- Para o dia a dia, prefira `docker compose start` ou `docker compose up -d`.
- Mantenha seu arquivo `docker-compose.yml` versionado.

Dica

Os dados do PostgreSQL (e outros serviços) ficam em volumes e **não** são removidos com o comando **down**, garantindo a **segurança** das suas informações.

"Com conhecimento, você tem mais controle sobre o seu ambiente."

Infocotidiano
CONHECIMENTO QUE TRANSFORMA

Infocotidiano

Estrutura do Projeto - PostgreSQL com Docker



SEU COMPUTADOR

Docker

Container: postgres-dev

PostgreSQL

banco: minha_api

postgres-data

Dados físicos do PostgreSQL

Estrutura de pastas

SEU COMPUTADOR

- Docker
  - Container: postgres-dev
    - PostgreSQL
      - banco: minha_api
- C:\Projetos\postgres-docker
  - docker-compose.yml
  - **postgres-data** ← dados físicos do PostgreSQL

Conhecimento que transforma

Tecnologia na prática

Infocotidiano